

Unit Testing

JUnit 5 & Mockito

Complete Senior Engineer Notes

80% Deep Theory · 20% Code · 70+ Interview Q&A

How It Works · Why It Works · When To Use It · Real Examples

Part 1: JUnit 5 · Part 2: Mockito · Full Practical Examples

Table of Contents

■■■■ PART 1 — JUnit 5 ■■■■

Chapter 1. Introduction to Unit Testing

- What is Unit Testing
- Why Testing Matters
- Key Principles: Independence, Repeatability, Isolation
- Test Pyramid Theory
- TDD — Test Driven Development

Chapter 2. Introduction to JUnit 5

- Architecture: Platform, Jupiter, Vintage
- JUnit 4 vs JUnit 5 Deep Comparison
- Maven/Gradle Setup
- How JUnit Discovers and Runs Tests

Chapter 3. JUnit 5 Annotations

- @Test Internals
- Lifecycle: @BeforeEach @AfterEach @BeforeAll @AfterAll
- @Disabled and @DisplayName
- Execution Order Theory

Chapter 4. Assertions in JUnit 5

- assertEquals / assertNotEquals
- assertTrue / assertFalse
- assertNull / assertNotNull
- assertAll — Soft Assertions
- assertThrows — Exception Testing

Chapter 5. Parameterized Tests

- @ValueSource @CsvSource @MethodSource
- When and Why to Parameterize
- Data-Driven Testing Theory

Chapter 6. Exception Handling in Tests

- assertThrows Deep Dive
- Testing Exception Messages
- Expected vs Unexpected Exceptions

■■■■ PART 2 — Mockito ■■■■

Chapter 7. Introduction to Mockito

- What is Mocking
- Mock vs Stub vs Spy vs Fake
- Why Mockito
- Dependency on External Systems

Chapter 8. Core Mockito Concepts

- Creating Mocks
- @Mock @InjectMocks @Captor
- Mockito Internals — Proxy Pattern

Chapter 9. Stubbing Behavior

- `when().thenReturn()`
- `thenThrow()` and `thenAnswer()`
- Argument Matchers
- Stubbing Void Methods

Chapter 10. Verifying Interactions

- `verify()` Deep Dive
- `Times`, `AtLeast`, `AtMost`, `Never`
- `verifyNoMoreInteractions`
- InOrder Verification

Chapter 11. Spies and Advanced Mockito

- `spy()` vs `mock()`
- `ArgumentCaptor` Theory and Usage
- Mocking Static Methods
- Best Practices

CHAPTER 1

Introduction to Unit Testing

Why isolating and testing individual units is the foundation of reliable software

1.1 The Story — Why Unit Testing Was Born

STORY

The year is 1994. Kent Beck, while working on the Smalltalk programming language, was frustrated. Large codebases were breaking unexpectedly. A change in one class would mysteriously break another class three layers away. Finding the source of a bug could take days. He created SUnit — the first unit testing framework — based on a simple idea: test each unit of code in complete isolation from everything else. When a test fails, you know EXACTLY which unit broke and why. This idea spread to Java as JUnit (by Kent Beck and Erich Gamma in 1997), then to Python, Ruby, JavaScript. Today, unit testing is considered as essential as the code itself.

1.2 What is Unit Testing — The Precise Definition

DEEP DIVE

A unit test verifies the behavior of the SMALLEST testable piece of code — typically a single method or class — in complete ISOLATION from its dependencies. "Complete isolation" is the critical phrase. If your method calls a database, a file system, an external API, or another class with complex behavior — those are replaced with controlled substitutes (mocks/stubs). This ensures the test is only measuring the correctness of YOUR code, not the correctness of the database driver or the network.

The word "unit" means different things in different contexts. In functional programming, a unit is a pure function. In object-oriented programming, a unit is typically a class or a single method. In microservices, a unit can be an entire service tested without its downstream dependencies. The key is that the boundary is small and well-defined.

1.3 The Three Key Principles of Unit Testing

Principle 1: Independence — No test should depend on the outcome of another test. Tests must be runnable in any order, any subset, any number of times. If Test A leaves some shared state that Test B relies on, and Test A fails, Test B will also fail even if its own logic is correct. This false failure wastes debugging time. In JUnit 5, a new instance of the test class is created for each test method — enforcing independence by design.

Principle 2: Repeatability — Given the same input, a test must produce the same result every single time. On your laptop, on the CI server, on your colleague's machine, at 3am or 3pm. Tests that rely on current time (new Date()), random numbers, or shared state are non-repeatable — they are called "flaky tests" and are worse than no tests at all because they train developers to ignore failures.

Principle 3: Isolation via Mocking — External dependencies (databases, HTTP APIs, file systems, message queues) are replaced with mock objects. This serves two purposes: (1) Speed — a test touching a real database takes seconds; a test with a mock takes milliseconds. Running 10,000 tests that each hit a database takes hours. With mocks, it takes seconds. (2) Control — with mocks you can simulate any scenario including database timeouts, network failures, and edge cases that would be impossible to reproduce with real systems.

1.4 The Test Pyramid — Where Unit Tests Fit

The Test Pyramid (introduced by Mike Cohn) describes the ideal proportion of different test types. Understanding this helps you allocate testing effort correctly.

Layer	Speed	Cost	Count	What It Tests
Unit Tests	Milliseconds	Cheapest	Many (70%)	Individual methods/classes in isolation with mocks
Integration Tests	Seconds	Medium	Some (20%)	Multiple components working together — real DB, real services
End-to-End Tests	Minutes	Most expensive	Few (10%)	Full application flow — browser, UI, all services

TIP

PRACTICAL INSIGHT: A common mistake is having an "ice cream cone" instead of a pyramid — many E2E tests, few unit tests. This makes the test suite slow, fragile and expensive to maintain. Unit tests should form the broad base. They run in seconds, catch most bugs, and pinpoint exactly where the problem is. Always write unit tests first.

1.5 Test Driven Development (TDD) — Theory

TDD (popularized by Kent Beck in "Test Driven Development: By Example", 2002) is a development process where you write the test BEFORE writing the production code. The cycle is: (1) RED — write a failing test for a feature that does not exist yet. (2) GREEN — write the minimum production code to make the test pass. (3) REFACTOR — clean up the code while keeping tests green. This forces you to think about the API and behavior of your code before implementation, leading to cleaner, more testable designs.

Interview Questions — Chapter 1

Q1. What is a unit test and what makes it different from an integration test?

Ans: A unit test verifies the behavior of a single method or class in complete isolation from its dependencies. All external dependencies (DB, APIs, other services) are replaced with mocks/stubs. It runs in milliseconds and precisely pinpoints the failing code. An integration test verifies that multiple components work correctly together — it uses real databases, real HTTP calls, real file systems. Integration tests are slower (seconds to minutes), harder to debug when they fail, but test realistic interactions. The rule of thumb: write many fast unit tests for all business logic, fewer integration tests for important integration points.

Q2. What is the Test Pyramid and why does it matter?

Ans: The Test Pyramid (Mike Cohn) says you should have many unit tests at the base, fewer integration tests in the middle, and very few E2E tests at the top. This is because unit tests are cheap (milliseconds, no infrastructure), catch most bugs, and pinpoint problems exactly. E2E tests are expensive (minutes, need full infrastructure), brittle (UI changes break them), and when they fail you need extensive debugging to find the root cause. Teams that ignore the pyramid end up with slow, flaky CI pipelines that developers stop trusting.

Q3. What is TDD and what are its benefits?

Ans: TDD (Test Driven Development) is a practice where you write a failing test BEFORE writing production code. Cycle: RED (write failing test) → GREEN (write minimum code to pass) → REFACTOR (clean up). Benefits: (1) Forces you to think about API design before implementation — tests act as the first client of your code. (2) Guarantees test coverage — every line of production code was written to satisfy a test. (3) Produces simpler, more modular code — code written to be testable is inherently loosely coupled. (4) Provides confidence to refactor — tests catch regressions immediately.

Q4. What makes a unit test "flaky" and how do you fix it?

Ans: A flaky test passes sometimes and fails sometimes without code changes. Common causes: (1) Dependency on system time (new Date()) — fix by injecting a Clock and mocking it. (2) Dependency on random numbers — fix by injecting a Random with a fixed seed or mocking it. (3) Shared mutable state between tests — fix by resetting state in @BeforeEach. (4) Test order dependency — one test leaves state that another relies on — fix by making each test set up its own complete state. (5) Thread timing — fix by using appropriate synchronization or single-threaded test executors. Flaky tests erode trust in the entire test suite.

CHAPTER 2

Introduction to JUnit 5

Architecture, modular design, how JUnit discovers and executes your tests

2.1 The Story — From JUnit 4 to JUnit 5

STORY

JUnit 4 (released 2006) was brilliant but had accumulated technical debt by 2015. It was a single monolithic JAR. Tool vendors (IDE makers, build tools) were tightly coupled to its internals. You could not mix different test libraries. Adding features required hacking the internal Runner API. A team of developers (led by Stefan Bechold, Sam Brannen, Johannes Link) started JUnit Lambda — which became JUnit 5 — completely rewritten from scratch. The design goal: separate the API for writing tests from the engine that runs them and the platform that launches everything. This modularity is the architectural breakthrough of JUnit 5.

2.2 JUnit 5 Architecture — Three Modules Explained

DEEP DIVE

JUNIT 5 = JUNIT PLATFORM + JUNIT JUPITER + JUNIT VINTAGE

JUnit Platform: The FOUNDATION. It is the launcher — the layer that IDEs (IntelliJ, Eclipse), build tools (Maven Surefire, Gradle), and CI systems use to discover and execute tests. It defines the TestEngine API which allows ANY test framework to plug in. This is what enables running JUnit 5, JUnit 4, TestNG, and Spock tests in the same build through their respective TestEngine implementations.

JUnit Jupiter: The PROGRAMMING MODEL for writing JUnit 5 tests. It is the new API: `@Test`, `@BeforeEach`, `@ParameterizedTest`, `@ExtendWith`, `Assertions` class — everything you write in your test files comes from Jupiter. Jupiter also includes a TestEngine implementation (JupiterTestEngine) that knows how to run Jupiter tests on the Platform.

JUnit Vintage: The BACKWARD COMPATIBILITY layer. It provides a TestEngine that can run JUnit 3 and JUnit 4 tests on the JUnit 5 Platform. This enables gradual migration — you can run old JUnit 4 tests alongside new JUnit 5 tests in the same Maven/Gradle build.

2.3 JUnit 4 vs JUnit 5 — Deep Comparison

Feature	JUnit 4	JUnit 5
Java version required	Java 5+	Java 8+ (lambdas, streams)
Architecture	Single monolithic JAR	Three modules: Platform, Jupiter, Vintage
Test annotation	<code>@org.junit.Test</code>	<code>@org.junit.jupiter.api.Test</code>
Setup per method	<code>@Before</code>	<code>@BeforeEach</code>
Teardown per method	<code>@After</code>	<code>@AfterEach</code>
Setup once per class	<code>@BeforeClass</code> (static only)	<code>@BeforeAll</code> (static by default, can use <code>@TestInstance</code>)
Teardown once	<code>@AfterClass</code> (static only)	<code>@AfterAll</code> (same as above)

Disable test	@Ignore	@Disabled (more descriptive)
Exception testing	@Test(expected=X.class)	assertThrows() — can also assert message
Parameterized tests	Separate @RunWith needed	Built-in @ParameterizedTest
Extension model	@RunWith (one only)	@ExtendWith (multiple, composable)
Nested tests	Not supported	@Nested — inner classes as test groups
Assertions	Assert class	Assertions class + third-party like AssertJ
Display names	Method name only	@DisplayName for readable test names

2.4 Maven Setup — What Each Dependency Does

```

<!-- pom.xml — Minimal JUnit 5 setup -->
<dependency>
<groupId>org.junit.jupiter</groupId>
<artifactId>junit-jupiter</artifactId> <!-- API + Engine -->
<version>5.10.2</version>
<scope>test</scope>
</dependency>
<!-- Maven Surefire Plugin -- REQUIRED to run JUnit 5 tests with mvn test -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version>
</plugin>
<!-- For Mockito add: -->
<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-junit-jupiter</artifactId>
<version>5.10.0</version>
<scope>test</scope>
</dependency>

```

WARNING

WHY SUREFIRE IS NEEDED: JUnit 5 changed how Maven discovers tests. Old Surefire (< 2.22) only knew about JUnit 4. You **MUST** use Surefire 2.22+ for JUnit 5 tests to be discovered and run during mvn test. Without this, maven test reports 0 tests run — a very common setup mistake.

2.5 How JUnit Discovers and Runs Tests — Internals

DEEP DIVE

TEST DISCOVERY AND EXECUTION FLOW: 1. Build tool (Maven/Gradle) or IDE invokes the JUnit Platform Launcher. 2. Platform uses ServiceLoader to discover all TestEngine implementations on the classpath (JupiterTestEngine, VintageTestEngine, etc.). 3. Each TestEngine is asked to discover tests matching the configured filter (class names, packages, tags). JupiterTestEngine scans for classes with @Test-annotated methods using reflection. 4. The Platform creates a TestPlan — a tree of discovered tests (engines → classes → test methods). 5. Platform executes the TestPlan: for each test class, JupiterTestEngine creates a new instance (by default, one per test method — this enforces test independence). 6. Lifecycle callbacks are invoked in order: @BeforeAll (static) → for each test: [@BeforeEach → @Test method → @AfterEach] → @AfterAll (static). 7. Assertions in the test method throw AssertionError if they fail. JupiterTestEngine catches this and records the test as FAILED. 8. Results flow back to the Platform, which reports them to the build tool and IDE.

Interview Questions — Chapter 2

Q1. What are the three modules of JUnit 5 and what does each do?

Ans: JUnit Platform is the foundation/launcher — it is the layer that IDEs, Maven, and Gradle use to discover and execute tests. It defines the TestEngine API that any framework can implement. JUnit Jupiter is the programming model for writing JUnit 5 tests — all annotations (`@Test`, `@BeforeEach`), Assertions class, and extensions API come from here. JUnit Vintage provides backward compatibility — it is a TestEngine implementation that runs JUnit 3 and 4 tests on the JUnit 5 Platform. This separation allows mixing different test frameworks in one build and enables IDEs to support JUnit 5 without being coupled to its internals.

Q2. Why does JUnit 5 create a new test class instance per test method?

Ans: This enforces test independence — the most important principle of unit testing. If one test modifies instance fields and the next test reuses the same instance, tests become order-dependent: passing or failing based on which tests ran before them. By creating a new instance per test, any instance field state set in one test cannot affect another. You can change this with `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` — then one instance is shared across all test methods in the class. Useful when setup is expensive (like opening a DB connection), but you must reset state manually in `@BeforeEach`.

Q3. What is the difference between `@Before` / `@After` in JUnit 4 and `@BeforeEach` / `@AfterEach` in JUnit 5?

Ans: They are functionally equivalent. `@Before` → `@BeforeEach`, `@After` → `@AfterEach`, `@BeforeClass` → `@BeforeAll`, `@AfterClass` → `@AfterAll`. The renaming in JUnit 5 makes the intent clearer: "Each" explicitly tells you this runs for every single test method. The biggest functional difference: in JUnit 4, `@BeforeClass` required a static method. In JUnit 5 with `@TestInstance(PER_CLASS)`, `@BeforeAll` can be non-static because a single instance is shared.

CHAPTER 3

JUnit 5 Annotations

@Test, lifecycle methods, @Disabled, @DisplayName — anatomy of a well-organized test class

3.1 The @Test Annotation — What It Really Does

@Test is the most fundamental JUnit 5 annotation. It marks a method as a test case that JupiterTestEngine should discover and execute. Internally, @Test is a meta-annotation — it is itself annotated with @Testable which is the marker the engine looks for during classpath scanning. A @Test method must be: non-private (JUnit 5 removed the public requirement — package-private works), non-static (unless using @TestInstance(PER_CLASS)), and return void. The method may throw any exception — if an uncaught exception escapes, the test FAILS.

3.2 The Complete Lifecycle — How Methods Execute

DEEP DIVE

COMPLETE EXECUTION ORDER FOR EVERY TEST METHOD: @BeforeAll (static, runs ONCE before all tests in the class start) ↓ [For each @Test method:] @BeforeEach (runs before THIS specific test) ↓ @Test method body executes ↓ @AfterEach (runs after THIS specific test — even if test threw exception) ↓ [Next @Test method...] ↓ @AfterAll (static, runs ONCE after all tests in the class complete) KEY INSIGHT: @AfterEach runs even if the @Test method threw an exception or failed an assertion. This guarantees cleanup happens regardless. Same for @AfterAll — it runs even if some tests failed. This is critical for releasing resources (closing files, releasing locks).

3.3 Each Annotation Explained In Depth

@BeforeEach — Setup Before Every Test

Runs before every single @Test method in the class. Use it to create fresh objects that tests need, set up mock behaviors, initialize variables. Since JUnit creates a new test class instance per test, @BeforeEach is the correct place to initialize instance fields. The goal: every test starts from a known, clean state. Never share mutable state between tests — that violates independence.

@AfterEach — Cleanup After Every Test

Runs after every @Test method, even if the test failed. Use it to close resources that were opened in @BeforeEach (file handles, database connections, network sockets). Also used to verify mock interactions that apply to all tests (though this is often done inside @Test methods with Mockito). In practice, @AfterEach is used less than @BeforeEach because JUnit creates a new instance per test, so object state is automatically discarded.

@BeforeAll — One-Time Expensive Setup

Runs ONCE before any test in the class. Must be static by default (because it runs before any instance is created). Use for expensive, time-consuming setup that is too slow to repeat per test: starting an embedded database, starting a WireMock server, loading large test data files. Since this is shared across all tests, it must be read-only — if tests modify the shared state, they become order-dependent.

@AfterAll — One-Time Cleanup

Runs ONCE after all tests complete. Must be static by default. Use to clean up resources created in @BeforeAll: stopping the embedded database, closing the WireMock server. Critical for CI environments where multiple test suites run in the same JVM.

```
// Complete example showing ALL lifecycle annotations
class OrderServiceTest {
    // Shared across all tests – expensive to create
    static EmbeddedDatabase db;
    // Fresh per test – lightweight objects
    OrderRepository repository;
    OrderService service;
    @BeforeAll
    static void startDatabase() {
        db = new EmbeddedDatabaseBuilder().setType(H2).build();
        System.out.println("DB started once");
    }
    @BeforeEach
    void setUp() {
        repository = new JdbcOrderRepository(db);
        service = new OrderService(repository);
        repository.deleteAll(); // Clean state before each test
    }
    @Test
    @DisplayName("Creating an order with valid data should return saved order with ID")
    void createOrder_validData_returnsSavedOrder() {
        Order order = new Order("PROD-1", 2, 99.99);
        Order saved = service.create(order);
        assertNotNull(saved.getId());
        assertEquals(2, saved.getQuantity());
    }
    @Test
    @Disabled("Skipped until payment gateway is mocked – see JIRA-456")
    void processPayment_integrationTest() {
        // TODO: requires payment gateway mock
    }
    @AfterEach
    void tearDown() {
        System.out.println("Cleanup after: " + testInfo.getDisplayName());
    }
    @AfterAll
    static void stopDatabase() {
        db.shutdown();
        System.out.println("DB stopped once");
    }
}
```

3.4 @DisplayName — Making Tests Readable

@DisplayName lets you give tests human-readable names that appear in IDE and CI reports. Without it, test reports show cryptic method names like "createOrder_validData_returnsSavedOrder". With @DisplayName("Creating an order with valid data should return a saved order with generated ID"), the report is self-documenting. This matters enormously in large teams — when a test fails in CI, the display name tells you exactly what broke without reading the code.

Interview Questions — Chapter 3

Q1. What is the execution order of JUnit 5 lifecycle methods?

Ans: Order: `@BeforeAll` (once, static) → then for each `@Test`: [`@BeforeEach` → `@Test` → `@AfterEach`] → `@AfterAll` (once, static). Critically: `@AfterEach` and `@AfterAll` run even if tests fail or throw exceptions — this guarantees cleanup. `@BeforeAll` and `@AfterAll` are static because they run before/after any test instance is created. With `@TestInstance(PER_CLASS)` they can be non-static.

Q2. When would you use `@BeforeAll` vs `@BeforeEach`?

Ans: `@BeforeAll`: for expensive one-time setup — starting an embedded database, loading large static test data, creating a WireMock server. Runs once; shared state must be read-only (or very carefully managed) to avoid test order dependencies. `@BeforeEach`: for cheap per-test setup — creating fresh object instances, resetting mock behaviors, initializing variables. Runs before every test, ensuring each test starts from a clean, known state. In unit tests with Mockito, `@BeforeEach` is used far more than `@BeforeAll`.

Q3. Why should `@BeforeAll` and `@AfterAll` methods be static in JUnit 5?

Ans: By default, JUnit 5 creates a new test class instance for each test method (`PER_METHOD` lifecycle). `@BeforeAll` runs before any instance exists, so it cannot be an instance method — it must be static. Similarly `@AfterAll` runs after all instances are garbage collected. If you use `@TestInstance(Lifecycle.PER_CLASS)`, a single instance is shared across all tests, and then `@BeforeAll/@AfterAll` can be non-static because the instance exists before the first test and after the last.

Q4. What is `@Disabled` and when should you use it?

Ans: `@Disabled` skips a test or an entire test class during execution. Use it when: a test is temporarily broken due to a known bug being fixed in parallel (always add a reason: `@Disabled("Fix pending — JIRA-456")`), a test depends on infrastructure not available in all environments, or a test is being rewritten. NEVER leave `@Disabled` tests without a reason or a ticket reference — they accumulate as dead code. Consider using `@EnabledOnOs`, `@EnabledIfSystemProperty`, or `@EnabledIfEnvironmentVariable` for conditional execution instead of blanket `@Disabled`.

CHAPTER 4

Assertions in JUnit 5

assertEquals, assertAll, assertThrows — verifying behavior with precision

4.1 What is an Assertion — The Theory

An assertion is a statement that a condition **MUST** be true at a specific point in your test. If the condition is false, the assertion throws `AssertionFailedError`, which `JupiterTestEngine` catches and records as a test failure. Assertions are the mechanism by which your test communicates: "I expected this, but got that." Without assertions, a test method that just runs your code without checking anything would always pass — it would be worse than useless because it provides false confidence.

INTERN

ASSERTION PHILOSOPHY: The assertion is your specification in code. `assertEquals(100, order.getTotal())` says "the total of this order **MUST** equal 100." If it does not, the test documents exactly what was expected vs what was actual. Good assertion messages tell a story: `assertNotNull(user.getId(), "Saved user must have a database-generated ID")`. The message appears when the assertion fails — make it count.

4.2 Core Assertions — Each One Explained

Assertion Method	What It Checks	When to Use
<code>assertEquals(expected, actual)</code>	<code>expected.equals(actual)</code> is true	Verify return values: prices, counts, strings, objects
<code>assertNotEquals(unexpected, actual)</code>	<code>expected.equals(actual)</code> is false	Verify that a value changed or is not a forbidden value
<code>assertTrue(condition)</code>	<code>condition == true</code>	Verify boolean results, <code>isEmpty()</code> , <code>contains()</code> , <code>matches()</code>
<code>assertFalse(condition)</code>	<code>condition == false</code>	Verify negative conditions
<code>assertNull(object)</code>	<code>object == null</code>	Verify optional return of null for missing items
<code>assertNotNull(object)</code>	<code>object != null</code>	Verify that an object was created/returned
<code>assertSame(expected, actual)</code>	<code>expected == actual</code> (reference)	Verify same object instance (rarely needed)
<code>assertThrows(type, executable)</code>	Executable throws specified exception	Test error paths — verify exceptions are thrown correctly
<code>assertAll(executables...)</code>	Runs all assertions, collects all failures	Soft assertions — see ALL failures not just the first
<code>assertIterableEquals(exp, act)</code>	Deep equality of iterables in order	Compare lists, sets element by element
<code>assertArrayEquals(exp, act)</code>	Array content equality	Compare arrays element by element

4.3 assertEquals — Deep Dive

`assertEquals` uses the `equals()` method to compare. This means: for primitives and their wrappers (`int`, `Integer`) it compares value. For `String` it compares character sequence. For custom objects it uses YOUR `equals()` method — if you have not overridden `equals()`, it falls back to reference equality (`==`), which almost always produces wrong results. Senior engineers override `equals()` and `hashCode()` in all DTO and entity classes.

```
// assertEquals examples
@Test
void calculateDiscount_premiumUser_gets20PercentOff() {
    User premium = new User("Alice", UserType.PREMIUM);
    double price = 100.0;
    double discounted = pricingService.applyDiscount(premium, price);
    assertEquals(80.0, discounted, 0.001, // delta for floating point!
        "Premium users should receive 20% discount");
    // String comparison
    assertEquals("PREMIUM", premium.getType().name());
    // WRONG for floating point – use delta version above:
    // assertEquals(80.0, discounted); // May fail due to floating point imprecision
}
```

4.4 assertAll — The Most Underused Assertion

By default, JUnit stops at the FIRST failed assertion in a test method. If you have 5 assertions and the first fails, you never see the results of assertions 2-5. This means one test run reveals one problem. `assertAll()` is the solution — it runs ALL provided assertions and collects ALL failures, reporting them together. This is called "soft assertions."

```
// Without assertAll – stops at first failure:
assertEquals("Alice", user.getName()); // If this fails...
assertEquals("alice@email.com", user.getEmail()); // ...this never runs
assertTrue(user.isActive()); // ...nor this
// With assertAll – runs ALL, reports ALL failures:
@Test
void createUser_allFieldsPopulatedCorrectly() {
    User user = userService.create(new UserDto("Alice", "alice@email.com"));
    assertAll("user fields validation",
        () -> assertNotNull(user.getId(), "ID must be generated"),
        () -> assertEquals("Alice", user.getName(), "Name must match input"),
        () -> assertEquals("alice@email.com", user.getEmail(), "Email must match"),
        () -> assertTrue(user.isActive(), "New user must be active"),
        () -> assertNotNull(user.getCreatedAt(), "Creation timestamp required")
    );
    // If name AND email both fail, you see BOTH failures in one run
}
```

4.5 assertThrows — Testing Error Paths

`assertThrows` is the JUnit 5 way to verify that a specific exception is thrown. It is far superior to the old JUnit 4 `@Test(expected=X.class)` approach because: (1) it returns the exception object so you can also verify the message, (2) it pinpoints WHICH line threw the exception, (3) it fails if the exception is thrown at the wrong point.

```
@Test
void findById_nonExistentId_throwsResourceNotFoundException() {
    // assertThrows returns the exception for further assertions
    ResourceNotFoundException ex = assertThrows(
        ResourceNotFoundException.class,
```

```
() -> userService.findById(999L), // This lambda must throw
"Should throw ResourceNotFoundException for non-existent ID"
);
// Verify the exception message too
assertTrue(ex.getMessage().contains("999"),
"Exception message should contain the missing ID");
}
@Test
void createUser_nullEmail_throwsIllegalArgumentException() {
assertThrows(IllegalArgumentException.class,
() -> userService.create(new UserDto("Alice", null)));
// Test PASSES because the exception WAS thrown
// Test FAILS if no exception is thrown (or wrong exception type)
}
```

Interview Questions — Chapter 4

Q1. What is the difference between assertEquals and assertEquals?

Ans: assertEquals uses the equals() method — it checks VALUE equality. Two different String objects with the same characters are assertEquals. assertEquals uses the == operator — it checks REFERENCE equality (same object in memory). Two different String objects with same characters are NOT assertEquals. In practice, assertEquals is almost never what you want — use assertEquals for value comparison. assertEquals is only useful when you specifically need to verify that the SAME cached/singleton object is being returned.

Q2. Why should you use assertEquals instead of multiple plain assertions?

Ans: Without assertEquals, JUnit stops at the FIRST failed assertion. If 3 of your 5 assertions fail, you only see the first failure in the test report — you must fix it, re-run, fix the next, re-run. This wastes developer time. assertEquals runs ALL assertions and collects ALL failures, showing them together. You can see the full picture in one run and fix everything at once. Use assertEquals whenever you are verifying multiple properties of the same object or operation.

Q3. How is assertEquals better than the old assertEquals(Exception.class) in JUnit 4?

Ans: Three improvements: (1) assertEquals returns the exception object — you can assert on its message, cause, and other properties. With assertEquals(expected=...) you only know the type was thrown, not what it said. (2) assertEquals scope: the lambda is precisely the code that should throw. With assertEquals(expected=...) ANY code in the method matching the exception type passes — including setup code that accidentally throws. (3) Readable: the intention is local and obvious in the lambda, not buried in the assertEquals annotation.

Q4. When should you use assertEquals(double, double, delta)?

Ans: Always for floating-point comparisons. Floating-point arithmetic (double, float) has inherent precision issues. $0.1 + 0.2 \neq 0.3$ in IEEE 754 arithmetic. assertEquals(0.3, 0.1+0.2) will FAIL because the actual result is 0.30000000000000004. The delta (tolerance) parameter: assertEquals(0.3, result, 0.0001) passes if the difference is less than 0.0001. Choose delta based on the precision your business logic requires. For currency: use BigDecimal and assertEquals without delta.

CHAPTER 5

Parameterized Tests

Data-driven testing with @ValueSource, @CsvSource, @MethodSource — test once, run with many inputs

5.1 The Problem Parameterized Tests Solve

STORY

STORY: A developer writes a test for an email validator. First: testValidEmail_withGmail. Then: testValidEmail_withYahoo. Then: testValidEmail_withOutlook. Then: testInvalidEmail_noAtSign. Then: testInvalidEmail_noDomain. Five test methods — each nearly identical. When the validator changes, you update ALL FIVE. When you think of a new test case, you write a SIXTH near-identical method. This is the combinatorial explosion problem. Parameterized tests solve it: one test method, many data inputs. Add a new case? Add one line of data.

5.2 @ParameterizedTest — How It Works Internally

@ParameterizedTest is a JUnit 5 meta-annotation that tells JupiterTestEngine: "run this method multiple times, once for each set of arguments provided by the ArgumentSource." The engine creates a separate test invocation for each argument set — each shows as a separate test in the IDE and CI report with the input value in the display name. This is the key benefit: you can see WHICH inputs pass and which fail.

5.3 @ValueSource — Simple Single Values

```
// @ValueSource — array of literals (one parameter per test)
@ParameterizedTest
@ValueSource(strings = {"alice@gmail.com", "bob@yahoo.co.in", "carol@company.org"})
void isValidEmail_validInputs_returnsTrue(String email) {
    assertTrue(emailValidator.isValid(email),
        email + " should be recognized as valid");
}
// Works with: strings, ints, longs, doubles, booleans, chars, classes
@ParameterizedTest
@ValueSource(ints = {1, 2, 5, 10, 100})
void isPositive_numbers_returnsTrue(int number) {
    assertTrue(MathUtils.isPositive(number));
}
```

5.4 @CsvSource — Multiple Parameters Per Test

```
// @CsvSource — multiple inputs + expected output in one test
@ParameterizedTest
@DisplayName("Discount calculation for different user tiers")
@CsvSource({
    "STANDARD, 100.0, 100.0", // No discount for standard
    "PREMIUM, 100.0, 80.0", // 20% off for premium
    "VIP, 100.0, 60.0", // 40% off for VIP
    "STANDARD, 0.0, 0.0", // Zero price edge case
})
void applyDiscount_differentTiers_correctPrice(
```



```
String tier, double price, double expected) {
    User user = new User(UserType.valueOf(tier));
    double result = pricingService.applyDiscount(user, price);
    assertEquals(expected, result, 0.001,
        tier + " user with price " + price + " should get " + expected);
}
```

5.5 @MethodSource — Dynamic and Complex Data

```
// @MethodSource — provide complex objects or large datasets
@ParameterizedTest
@MethodSource("provideOrdersForTotalCalculation")
void calculateTotal_variousOrders_correctTotal(Order order, double expectedTotal) {
    assertEquals(expectedTotal, orderService.calculateTotal(order), 0.01);
}
// The method must be static and return Stream
static Stream provideOrdersForTotalCalculation() {
    return Stream.of(
        Arguments.of(new Order(List.of(item(10.0), item(20.0))), 30.0),
        Arguments.of(new Order(List.of(item(5.0))), 5.0),
        Arguments.of(new Order(Collections.emptyList()), 0.0),
        Arguments.of(new Order(List.of(item(99.99), item(0.01))), 100.0)
    );
}
// Benefits: can create real objects, load from files/DB, generate dynamically
```

Interview Questions — Chapter 5

Q1. When should you use @ValueSource vs @CsvSource vs @MethodSource?

Ans: @ValueSource: single-parameter tests with literal values (strings, ints, etc.). Simple, readable, inline. Best for: testing a validation function with many valid or invalid inputs. @CsvSource: multiple parameters per invocation as inline CSV strings. Best for: input-output pairs like (input, expected), (tier, price, discount). Readable inline, good for up to 10-15 rows. @MethodSource: when you need complex objects (cannot express as CSV strings), large datasets (loaded from files/DB), or dynamic generation. A separate static factory method returns Stream. Most flexible but adds indirection.

Q2. What is displayed in the test report for a parameterized test?

Ans: Each parameterized test invocation appears as a separate test entry in the IDE and CI report. The display name shows the index and the argument values by default: "[1] alice@gmail.com", "[2] bob@yahoo.co.in". You can customize with @ParameterizedTest(name = "Test {index}: {0} should be valid"). This is one of the biggest benefits of parameterized tests over data-in-loop approaches — you see exactly which input failed, not just that "the loop test failed."

Q3. How do you parameterize a test with a null value?

Ans: @ValueSource does not directly support null. Use @NullSource (adds one null invocation), @EmptySource (adds one empty string/collection invocation), or @NullAndEmptySource (adds both). Combine with @ValueSource using @CombineWith or nest them. For complex nullability testing: use @MethodSource with Arguments.of((Object)null, expectedValue). Always test null separately — it is a common source of NullPointerExceptions in production.

CHAPTER 6

Exception Handling in Tests

Testing error paths completely — assertThrows, exception messages, expected vs unexpected

6.1 Why Testing Exception Paths Matters

STORY

A common mistake: developers test only the "happy path" — the scenario where everything goes right. But production systems spend significant time in error paths. A user provides invalid data. A network call times out. A database constraint is violated. A file is not found. These error paths have business logic: the correct exception type must be thrown, with the correct message, and the system must remain in a consistent state after the error. Without tests for error paths, this code is unverified and defects hide there until production.

6.2 Complete Exception Testing Strategy

```
// STRATEGY 1: Verify exception TYPE
@Test
void withdraw_insufficientFunds_throwsException() {
    Account account = new Account(50.0); // Only 50 rupees
    assertThrows(InsufficientFundsException.class,
        () -> account.withdraw(100.0));
}

// STRATEGY 2: Verify exception TYPE + MESSAGE
@Test
void findUser_notFound_throwsWithCorrectMessage() {
    long missingId = 999L;
    InsufficientFundsException ex = assertThrows(
        InsufficientFundsException.class,
        () -> bankService.withdraw(missingId, 100.0)
    );
    assertTrue(ex.getMessage().contains("999"),
        "Exception should mention the account ID");
    assertTrue(ex.getMessage().contains("insufficient"),
        "Exception should explain the reason");
}

// STRATEGY 3: Verify system state AFTER exception
@Test
void save_duplicateEmail_throwsAndDoesNotSaveAnything() {
    userRepo.save(new User("alice@email.com")); // First save
    assertThrows(DuplicateEmailException.class,
        () -> userService.register(new UserDto("alice@email.com")));
    // Verify state is consistent – still only ONE user
    assertEquals(1, userRepo.count(),
        "Failed save should not persist partial data");
}

// ANTI-PATTERN: Do NOT use try-catch in tests
// @Test void bad_test() {
//     try { service.doSomething(); fail("Should have thrown"); }
//     catch (Exception e) { /* This is what assertThrows replaces */ }
// }
```

Interview Questions — Chapter 6

Q1. How do you test that a method throws the RIGHT exception with the RIGHT message?

Ans: Use `assertThrows` which returns the exception object. Then assert on the returned exception: `ResourceNotFoundException ex = assertThrows(ResourceNotFoundException.class, () -> service.findById(999L));` followed by `assertTrue(ex.getMessage().contains("999"))` and `assertTrue(ex.getMessage().contains("not found"))`. This two-step approach lets you verify both the exception type AND its message, ensuring your error handling is precise and informative — not just that something was thrown.

Q2. What is the anti-pattern for exception testing in JUnit?

Ans: The old try-catch pattern: `try { service.doThing(); fail("Should have thrown"); } catch (ExpectedException e) { /* test passes */ } catch (Exception e) { fail("Wrong exception type"); }`. Problems: (1) Verbose and noisy. (2) Easy to forget the `fail()` call — if the method does NOT throw, the test passes silently (false positive). (3) Cannot easily assert on the exception message. `assertThrows` eliminates all these issues — it is shorter, cannot produce false positives, and returns the exception for further assertions.

CHAPTER 7

Introduction to Mockito

What is mocking, why it matters, Mock vs Stub vs Spy vs Fake — the complete vocabulary

7.1 The Story — Testing Without Mocking is a Nightmare

STORY

A senior developer at a fintech startup needs to test the interest calculation logic in `LoanService`. The logic is pure math — straightforward. But `LoanService` has dependencies: it calls a `CreditScoreAPI` (external HTTP service), a `DatabaseRepository` (MySQL), and an `AuditLogger` (writes to files). To test the interest calculation *WITHOUT* mocking: you need a running database, a live credit score API (rate-limited, costs money per call), and a writable file system. Running 500 tests takes 15 minutes. The CI server has no credit score API access, so tests fail in CI but pass locally. Debugging takes hours. With Mockito: `CreditScoreAPI` is mocked to return any score you need, `DatabaseRepository` is mocked to return predefined users, `AuditLogger` is mocked to do nothing. 500 tests run in 2 seconds. CI passes instantly.

7.2 Mock vs Stub vs Spy vs Fake — The Precise Vocabulary

These terms are used interchangeably in casual conversation but have distinct meanings in testing theory (Gerard Meszaros, "xUnit Test Patterns"):

Term	Definition	What It Returns	Verification
Mock	A test double that records interactions. You can verify later that specific methods were called.	Configured values or defaults (0, null, empty)	Yes — <code>verify()</code> checks what was called and how many times
Stub	A test double that returns pre-configured responses. Does not verify interactions.	Pre-configured (stubbed) values	No — only provides data
Spy	A wrapper around a REAL object. Real methods are called UNLESS specifically overridden.	Real return values by default	Yes — can verify what was called
Fake	A simplified working implementation (e.g., in-memory database instead of MySQL).	Real computed values	No
Dummy	A placeholder object passed as an argument but never used.	Anything (not called)	No

INTERN

IN MOCKITO: Mockito.mock() creates objects that are BOTH mocks AND stubs. They can return pre-configured values (stubbing behavior) AND record interactions for later verification (mock behavior). Mockito.spy() creates a spy. The distinction Mock vs Stub is conceptual — in Mockito code, the same object can be used as either depending on whether you call when().thenReturn() (stubbing) or verify() (mocking).

7.3 Why Mockito — The Benefits In Depth

- **Speed:** Mock objects return values instantly — no network, no disk, no database. A suite of 10,000 tests that would take 30 minutes with real dependencies runs in 30 seconds with mocks.
- **Determinism:** External services return different things at different times (rate limits, data changes, network variability). Mocks always return exactly what you configure — tests are repeatable.
- **Error Simulation:** You cannot easily make a real database throw a connection timeout to test your retry logic. With mocks: when(repo.findById(any())).thenThrow(new DataAccessException("timeout")) — one line.
- **Isolation:** When a test fails with mocks, you know the bug is in YOUR code (the class under test), not in a dependency. Without mocks, a test failure could be in any of the 5 classes involved.
- **Simplicity:** Mocking a complex object with 20 methods and only needing 2 of them — define only those 2 behaviors. You do not need to set up the entire system.

Interview Questions — Chapter 7

Q1. What is the difference between a mock and a stub?

Ans: A stub provides pre-configured return values — it is a data provider. You configure it: when called with this input, return that output. No verification. A mock records all interactions — you can verify at the end: was this method called? How many times? With which arguments? In Mockito, Mockito.mock() creates objects that can behave as both. when().thenReturn() is stubbing behavior. verify() uses the mock/recording behavior. The conceptual distinction matters for test design: stubs are for setting up test data, mocks are for verifying behavior (specifically, verifying interactions with dependencies).

Q2. What is a spy in Mockito and when do you use it?

Ans: A spy wraps a real object. By default, all method calls go to the REAL implementation. You can selectively stub specific methods: doReturn(x).when(spy).specificMethod(). Use spy when: (1) You want to test the real implementation but override just one problematic method (e.g., a method that calls an external service). (2) You need to verify interactions on a real object. (3) Migrating legacy code to be testable gradually. Prefer pure mocks over spies when possible — spies can cause confusion because some calls are real and some are mocked.

Q3. When should you NOT use mocking?

Ans: Do not mock: (1) Value objects / DTOs — simple data holders with no behavior. (2) Utility classes (String, List, LocalDate) — they are pure functions, deterministic, no side effects. (3) The class you are testing (the SUT — System Under Test) — that defeats the purpose. (4) Every single collaborator when the interaction is simple and the collaborator is a lightweight object. Over-mocking makes tests brittle: changing a method signature breaks many tests even if the behavior did not change. Mock only external dependencies and collaborators with complex behavior or side effects.

CHAPTER 8

Core Mockito Concepts

Creating mocks, @Mock, @InjectMocks, @Captor — annotations and their internal mechanisms

8.1 Creating Mocks — Two Ways

Mockito provides two approaches: programmatic (Mockito.mock()) and annotation-driven (@Mock). The annotation approach is strongly preferred in real projects because it is less verbose, automatically tied to the test class lifecycle, and produces better error messages.

```
// WAY 1: Programmatic (verbose, use only for quick one-offs)
UserRepository repo = Mockito.mock(UserRepository.class);
EmailService emailSvc = Mockito.mock(EmailService.class);
UserService service = new UserService(repo, emailSvc); // Manual wiring
// WAY 2: Annotations (preferred – cleaner, automatic)
@ExtendWith(MockitoExtension.class) // REQUIRED for annotations to work
class UserServiceTest {
    @Mock
    UserRepository userRepository; // Mockito creates mock automatically
    @Mock
    EmailService emailService; // Another mock
    @InjectMocks
    UserService userService; // Mockito injects mocks into this
    @Captor
    ArgumentCaptor userCaptor; // For capturing method arguments
    @Test
    void registerUser_validData_savesAndSendsEmail() {
        // Mocks are ready – no manual creation needed
        when(userRepository.save(any())).thenReturn(new User(1L, "Alice"));
        // ... test code
    }
}
```

8.2 @ExtendWith(MockitoExtension.class) — Why It Is Required

DEEP DIVE

MockitoExtension is a JUnit 5 extension that hooks into the test lifecycle. BEFORE each test: it scans the test class for @Mock, @Spy, @Captor, @InjectMocks fields and initializes them. This is equivalent to calling MockitoAnnotations.openMocks(this) in @BeforeEach. AFTER each test: it validates that all stubbed interactions were actually used (detects unnecessary stubbings that indicate test code rot). Without @ExtendWith(MockitoExtension.class), annotations are NOT processed — your @Mock fields remain null and you get NullPointerException in every test.

8.3 @InjectMocks — How Injection Works Internally

@InjectMocks tells Mockito to create an instance of the specified class and inject all @Mock and @Spy fields into it. Mockito tries three injection strategies IN ORDER until one succeeds:

Strategy	How It Works	When Mockito Uses It
----------	--------------	----------------------

Constructor Injection	Finds the largest constructor and passes @Mock fields as arguments	Preferred — if a matching constructor exists
Setter Injection	Calls setter methods (setXxx()) for each @Mock field	If no constructor injection possible and setters exist
Field Injection	Directly sets private fields via reflection	Last resort — works but hides coupling

TIP

BEST PRACTICE: Design your production classes with constructor injection. This is the cleanest approach — your dependencies are explicit, required (cannot be null), and clearly visible. With constructor injection, @InjectMocks works perfectly and your class is testable WITHOUT Mockito (just pass mocks as constructor arguments manually). Classes that use field injection with @Autowired are harder to test because dependencies are hidden.

Interview Questions — Chapter 8

Q1. What does @InjectMocks do and how does it decide which injection strategy to use?

Ans: @InjectMocks tells Mockito to create an instance of the SUT (System Under Test) class and inject the @Mock/@Spy fields into it. Mockito tries three strategies in order: (1) Constructor injection — finds the largest constructor that matches the mock types and calls it with the mocks as arguments. This is the preferred strategy. (2) Setter injection — looks for setter methods matching the mock types and calls them. (3) Field injection — directly sets private fields via reflection. If no strategy works, the object is created with a no-arg constructor and mocks remain uninjected (silent failure — a common bug).

Q2. What happens if you forget @ExtendWith(MockitoExtension.class)?

Ans: Annotations (@Mock, @InjectMocks, @Captor) are NOT processed. Fields annotated with @Mock remain null (Java default for reference types). When your test method accesses userRepository or tries to use userService (which itself has null dependencies), you get NullPointerException. The error message points to your test code, not to the missing extension, making it confusing to debug. Solution: always add @ExtendWith(MockitoExtension.class) at the class level for any test class using Mockito annotations.

Q3. What is the difference between Mockito.mock() and @Mock annotation?

Ans: Functionally equivalent — both create a mock of the specified type. @Mock is annotation-driven and cleaner: less boilerplate, better integrated with the test lifecycle (automatically reset between tests by MockitoExtension), and produces clearer test reports (mocks have descriptive names based on field names). Mockito.mock() is programmatic — useful when you need to create mocks conditionally, inside loops, or in non-test helper classes. In test classes, always prefer @Mock for readability.

CHAPTER 9

Stubbing Behavior

when().thenReturn(), thenThrow(), thenAnswer(), argument matchers, void methods

9.1 Stubbing — The Theory

Stubbing is configuring a mock to return a specific value or throw a specific exception when called with specific arguments. Without stubbing, a mock method returns: 0 for int/long/double, false for boolean, empty collection for List/Set/Map, and null for all object types. These defaults are fine for tests where you do not care about the return value, but when your code actually uses the return value, you must stub it.

9.2 when().thenReturn() — The Core Stubbing Pattern

```
@Test
void getUserById_existingUser_returnsUserDto() {
    // ARRANGE – set up what the mock should return
    User fakeUser = new User(42L, "Alice", "alice@email.com");
    when(userRepository.findById(42L)) // When this is called with argument 42...
        .thenReturn(Optional.of(fakeUser)); // ...return this value
    // ACT – call the real service method
    UserDto result = userService.getUserById(42L);
    // ASSERT – verify the result
    assertEquals("Alice", result.getName());
    assertEquals("alice@email.com", result.getEmail());
}

// Chain multiple returns – first call returns X, second call returns Y
when(service.getNextItem())
    .thenReturn("first")
    .thenReturn("second")
    .thenReturn("third"); // Subsequent calls return "third" forever
```

9.3 thenThrow() — Simulating Failures

```
// Simulate database connection failure
@Test
void getUserById_databaseDown_throwsServiceException() {
    when(userRepository.findById(anyLong()))
        .thenThrow(new DataAccessException("Connection timeout"));
    // Your service should wrap low-level exceptions:
    ServiceException ex = assertThrows(ServiceException.class,
        () -> userService.getUserById(1L));
    assertEquals("Database unavailable, please try again", ex.getMessage());
}

// Simulate FIRST call fails, SECOND call succeeds (retry logic testing)
when(externalApi.fetchData())
    .thenThrow(new NetworkException("Timeout"))
    .thenReturn(new ApiResponse("success"));
```

9.4 Argument Matchers — Flexible Stubbing

By default, Mockito uses `equals()` to match arguments — `when(repo.findById(42L))` only matches when the argument is EXACTLY 42L. Argument matchers provide flexibility: `any()`, `anyLong()`, `anyString()`, `eq()`, `argThat(lambda)`. Important rule: if you use matchers for ANY argument, you must use matchers for ALL arguments.

```
// any() – match any argument of compatible type
when(userRepository.save(any(User.class))).thenReturn(savedUser);
// anyString(), anyLong(), anyInt(), anyList()...
when(emailService.send(anyString(), anyString())).thenReturn(true);
// eq() – match exact value (needed when mixing matchers)
when(repo.findByNameAndAge(eq("Alice"), anyInt())).thenReturn(user);
// WRONG: when(repo.findByNameAndAge("Alice", anyInt())) -- mix not allowed
// argThat() – custom matcher with lambda
when(repo.save(argThat(u -> u.getEmail().endsWith("@company.com"))))
    .thenReturn(approvedUser);
// For void methods – use doReturn / doThrow / doNothing BEFORE method
doNothing().when(auditLogger).log(anyString());
doThrow(new AuditException()).when(auditLogger).log("FORBIDDEN_ACTION");
```

9.5 thenAnswer() — Dynamic Responses

```
// thenAnswer – compute return value based on actual argument
when(userRepository.findById(anyLong()))
    .thenAnswer(invocation -> {
        Long id = invocation.getArgument(0);
        if (id < 0) return Optional.empty();
        return Optional.of(new User(id, "User" + id));
    });
// Useful when: return value depends on input, simulating a real repository
```

Interview Questions — Chapter 9

Q1. What is the difference between `thenReturn()`, `throw()`, and `thenAnswer()`?

Ans: `thenReturn(value)`: always returns the given value (or null if value is null). Simple and most common. `throw(exception)`: throws the exception when the method is called — use for testing error handling, retry logic, and failure scenarios. `thenAnswer(invocation -> ...)`: computes the return value dynamically based on the actual arguments received. Use when the return value depends on the input (simulating ID-based lookups, echo services, etc.). You can chain them: `thenReturn(x).throw(e).thenReturn(y)` for testing retry logic.

Q2. When can you NOT use `when().thenReturn()` and must use `doReturn().when()`?

Ans: Two cases: (1) Void methods: `when(mock.voidMethod())` does not compile because there is no return value to capture. Use `doNothing().when(mock).voidMethod()` or `doThrow().when(mock).voidMethod()`. (2) Spies with real implementations: `when(spy.realMethod())` actually CALLS the real method before stubbing it (before the stubbing takes effect). If the real method has side effects, this causes issues. Use `doReturn(value).when(spy).realMethod()` which does NOT call the real method.

Q3. What happens if you stub a method that is never called?

Ans: By default, with MockitoExtension (@ExtendWith(MockitoExtension.class)), Mockito detects unnecessary stubbings and throws UnnecessaryStubbingException after the test. This is called "strict stubbing." It helps keep tests clean — if you stub when(repo.findById(1L)).thenReturn(user) but the test never calls repo.findById(), it is dead code in your test. Fix: remove the unnecessary stub. If you need to allow unused stubs (legacy code, shared setup): use @MockitoSettings(strictness=Strictness.LENIENT) at the class level.

CHAPTER 10

Verifying Interactions

verify() internals, times(), inOrder, verifyNoMoreInteractions — ensuring correct behavior

10.1 Why Verify? — The Theory of Interaction Testing

Assertions verify the RETURN VALUE of operations. But some operations have no return value — void methods. And some behaviors cannot be fully verified through return values alone. Example: `orderService.placeOrder()` returns an `Order`. But does it also call `emailService.sendConfirmation()`? Does it call `auditLogger.log()`? You cannot verify these from the return value. `verify()` fills this gap — it answers: "was this dependency method called, how many times, and with what arguments?"

INTERN

WHEN TO VERIFY vs WHEN TO ASSERT: Verify void methods and side-effect behaviors: sending emails, logging, publishing events, calling notification services. Assert return values: `calculateTotal()`, `getUser()`, `isValid()`. Do NOT verify trivial getters or interactions that are implicit in your assertions. Over-verification makes tests brittle — internal implementation changes break tests even when behavior is correct.

10.2 verify() — Complete Usage

```
@Test
void placeOrder_validOrder_savesAndSendsConfirmation() {
    Order order = new Order("PROD-1", 2);
    when(orderRepo.save(any())).thenReturn(new Order(1L, "PROD-1", 2));
    orderService.placeOrder(order);
    // Verify save was called exactly ONCE with any Order
    verify(orderRepo, times(1)).save(any(Order.class));
    // Verify email was sent (void method – can only verify, not assert)
    verify(emailService, times(1)).sendConfirmation(eq("PROD-1"), anyString());
    // Verify audit was logged TWICE (create + confirm)
    verify(auditLogger, times(2)).log(anyString());
    // Verify that NO other methods were called on these mocks
    verifyNoMoreInteractions(emailService, auditLogger);
}

// Verification modes:
verify(mock); // Exactly once (default)
verify(mock, times(3)); // Exactly 3 times
verify(mock, atLeast(1)); // At least once
verify(mock, atLeastOnce()); // Same as atLeast(1)
verify(mock, atMost(5)); // At most 5 times
verify(mock, never()); // Was NEVER called – critical for security
```

10.3 InOrder — Verifying Execution Sequence

```
// Verify that operations happen in the CORRECT ORDER
@Test
void processPayment_correctOrderOfOperations() {
    paymentService.processPayment(order, creditCard);
    InOrder inOrder = inOrder validator, paymentGateway, receiptService);
    inOrder.verify(validator).validate(creditCard); // Must happen FIRST
```

```
inOrder.verify(paymentGateway).charge(any()); // Then charge
inOrder.verify(receiptService).generateReceipt(); // Then receipt
// If charge() happens before validate(), this test FAILS
}
```

Interview Questions — Chapter 10

Q1. When should you use verify() and when should you NOT?

Ans: Use verify() for: (1) Void methods — the only way to test they were called. (2) Side effects: sending emails, publishing events, calling external services. (3) Security-critical non-calls: verify(paymentProcessor, never()).charge() if validation fails. Do NOT use verify() for: (1) Methods whose return value you are already asserting — assertEquals(expected, result) already proves the method was called and returned the right value. (2) Trivial getters in the SUT. (3) Every single interaction — over-verification makes tests break on refactoring even when behavior is correct.

Q2. What is verifyNoMoreInteractions() and should you use it?

Ans: verifyNoMoreInteractions(mock) asserts that NO methods were called on the mock beyond what you have already explicitly verified. It is useful for detecting unexpected interactions — your code is calling something it should not. However, it is controversial: it makes tests very brittle because any new call to the mock (even benign ones) breaks the test, even if the behavior is still correct. Recommendation: use it sparingly, only for critical security or billing scenarios where unexpected calls would be dangerous. For most tests, verify only the interactions you care about.

Q3. What is InOrder verification and when is it needed?

Ans: InOrder verifies that multiple mock interactions happened in a specific sequence. Create: InOrder inOrder = inOrder(mock1, mock2). Then: inOrder.verify(mock1).method1(); inOrder.verify(mock2).method2(). If the actual execution order was reversed, the test fails. Use for: payment processing workflows (validate → charge → receipt), transaction management (begin → execute → commit or rollback), state machine transitions. Without InOrder, regular verify() does not care about order — it only checks that each call happened, not when.

CHAPTER 11

Spies, ArgumentCaptor & Best Practices

spy() internals, ArgumentCaptor deep dive, advanced patterns, the senior engineer checklist

11.1 Spies — Real Objects with Selective Override

A Mockito spy wraps a real object. All method calls by default go to the REAL implementation and return real values. You can selectively override specific methods using `doReturn().when(spy).method()` syntax. Spies are useful when you want to test real behavior but need to isolate one problematic dependency method.

```
// Spy on a REAL object (not an interface)
List realList = new ArrayList<>();
List spiedList = spy(realList);
// Real methods work:
spiedList.add("hello");
assertEquals(1, spiedList.size()); // Returns 1 – real implementation
// Override specific method:
doReturn(42).when(spiedList).size(); // Now size() returns 42
assertEquals(42, spiedList.size());
// IMPORTANT: Use doReturn (NOT when().thenReturn()) for spies
// when(spiedList.size()).thenReturn(42) // CALLS real size() first – side effects!
// Practical use: testing a service that calls its OWN helper method
@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {
    @Spy
    NotificationService service = new NotificationService();
    @Test
    void sendAlert_callsInternalFormatter() {
        doReturn("FORMATTED").when(service).formatMessage(anyString());
        service.sendAlert("raw message");
        verify(service).formatMessage("raw message");
    }
}
```

11.2 ArgumentCaptor — Capturing and Asserting Arguments

ArgumentCaptor captures the actual argument passed to a mock method and lets you assert on it. This is invaluable when you cannot easily assert on the return value but need to verify what exactly was passed to a dependency.

```
@Test
void registerUser_sendsWelcomeEmailWithCorrectContent() {
    // We want to verify WHAT email was sent, not just THAT it was sent
    userService.register(new UserDto("Alice", "alice@email.com"));
    // Capture the Email object passed to emailService.send()
    ArgumentCaptor emailCaptor = ArgumentCaptor.forClass(Email.class);
    verify(emailService).send(emailCaptor.capture());
    Email sentEmail = emailCaptor.getValue();
    assertEquals("alice@email.com", sentEmail.getTo());
    assertEquals("Welcome to Our Platform!", sentEmail.getSubject());
    assertTrue(sentEmail.getBody().contains("Alice"),
        "Email body should be personalized with user name");
}
```

```
// Using @Captor annotation (cleaner)
@Captor ArgumentCaptor emailCaptor;
// Capture MULTIPLE calls:
verify(emailService, times(2)).send(emailCaptor.capture());
List allEmails = emailCaptor.getAllValues();
assertEquals("alice@email.com", allEmails.get(0).getTo());
assertEquals("admin@company.com", allEmails.get(1).getTo());
```

11.3 Mockito Best Practices — The Senior Engineer Checklist

- **Mock only external dependencies** — databases, HTTP clients, file systems, message queues. Do not mock value objects, DTOs, or simple collaborators.
- **Use constructor injection** — it makes @InjectMocks work cleanly and your class is testable without Mockito at all.
- **Use strict stubbing** (Mockito default with MockitoExtension) — catches unnecessary stubbings and keeps tests clean.
- **Do not use @InjectMocks with field injection classes** — field injection hides dependencies. Refactor to constructor injection first.
- **Reset mocks with care** — if you MUST share a mock (e.g., in @BeforeAll), call Mockito.reset(mock) in @BeforeEach. But prefer creating fresh mocks per test.
- **Avoid over-verification** — verify only meaningful, non-obvious interactions. Do not verify every single method call.
- **Use descriptive display names** with @DisplayName so CI reports are readable when tests fail.
- **Follow AAA pattern** — Arrange (set up mocks and data), Act (call the method under test), Assert (verify return value and interactions). Keep these three phases visually separated.
- **Test one thing per test** — one logical behavior, one reason to fail, one assertion group. A test with 20 assertions is really 5 tests merged together.
- **Never mock the class under test** — mocking the SUT defeats the purpose. You would be testing the mock, not your code.
- **Name tests with the pattern: methodName_scenario_expectedBehavior** — e.g., placeOrder_insufficientStock_throwsOutOfStockException.

```
// COMPLETE EXAMPLE — Everything Together (AAA pattern)
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock UserRepository userRepo;
    @Mock ProductRepository productRepo;
    @Mock EmailService emailService;
    @Mock AuditLogger auditLogger;
    @InjectMocks OrderService orderService; // Constructor-injected
    @Captor ArgumentCaptor emailCaptor;
    @Test
    @DisplayName("Place order: valid data saves order and sends confirmation email")
    void placeOrder_validData_savesAndSendsConfirmation() {
        // ARRANGE
        User alice = new User(1L, "Alice", "alice@email.com");
        Product product = new Product("PROD-1", "Laptop", 999.99, 10);
        when(userRepo.findById(1L)).thenReturn(Optional.of(alice));
        when(productRepo.findById("PROD-1")).thenReturn(Optional.of(product));
        when(userRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
        // ACT
        Order result = orderService.placeOrder(1L, "PROD-1", 2);
```

```
// ASSERT – return value
assertAll("order fields",
() -> assertNotNull(result),
() -> assertEquals(1999.98, result.getTotal(), 0.01),
() -> assertEquals(OrderStatus.CONFIRMED, result.getStatus())
);
// ASSERT – interactions
verify(emailService).send(emailCaptor.capture());
Email sentEmail = emailCaptor.getValue();
assertEquals("alice@email.com", sentEmail.getTo());
assertTrue(sentEmail.getSubject().contains("Order Confirmed"));
verify(auditLogger).log(contains("ORDER_PLACED"));
verify(productRepo, never()).deleteById(any()); // Stock not deleted
}
```

Interview Questions — Chapter 11

Q1. What is the difference between mock() and spy() in Mockito?

Ans: mock(Class.class) creates a FAKE object. By default ALL methods return empty values (null, 0, false, empty collections). You stub only what you need. spy(realObject) wraps a REAL instance. By default ALL methods call the REAL implementation. You stub only the methods you need to override. Use mock() (the vast majority of the time) when testing against interfaces or when you want full control. Use spy() when testing a class that calls its own internal methods that you need to isolate, or when migrating legacy untestable code.

Q2. What is ArgumentCaptor and when is it better than argument matchers?

Ans: ArgumentCaptor captures the actual argument passed to a mock method after the call, for assertions. Use it when you need to: (1) Verify the CONTENT of a complex object passed to a void method (email content, event payload, message body). (2) Assert on computed values (like generated IDs or timestamps) passed to dependencies. Argument matchers (any(), eq()) are for STUBBING — defining behavior based on inputs. Captors are for VERIFICATION — inspecting what was actually passed. The two serve different purposes in the test.

Q3. What is strict stubbing in Mockito and why is it a good default?

Ans: Strict stubbing (enabled by default with MockitoExtension) detects: (1) Unnecessary stubbings — when() definitions that are never called during the test. These indicate test code rot — leftover setup from a refactoring. (2) Stubbing argument mismatches — when you stub with one argument but call with another. With LENIENT mode, these issues are silently ignored. Strict stubbing makes tests self-maintaining: they fail when they contain dead code. This is a best practice — always use strict stubbing.

Q4. Explain the complete flow of a well-written Mockito test from setup to assertion.

Ans: A complete test follows AAA: ARRANGE: Create mock dependencies with @Mock, create SUT with @InjectMocks, configure mock behavior with when().thenReturn() for values the SUT will use. ACT: Call the single method under test on the SUT with the test input. ASSERT: (1) Assert on the return value using assertEquals/assertAll. (2) Assert on side effects by capturing arguments with ArgumentCaptor and asserting on them. (3) Verify critical interactions with verify(). (4) Optionally verify non-interactions with verify(mock, never()).method(). Each test should test ONE behavior — one @Test per scenario.

Quick Reference Appendix

JUnit 5 Annotations — Complete Reference

Annotation	Purpose	Key Notes
@Test	Marks method as test case	Non-private, non-static (default), returns void
@BeforeEach	Runs before every @Test method	Use for per-test setup — fresh objects, mock config
@AfterEach	Runs after every @Test (even on failure)	Use for per-test cleanup — close resources
@BeforeAll	Runs ONCE before all tests (static)	Expensive one-time setup — DB start, WireMock
@AfterAll	Runs ONCE after all tests (static)	One-time cleanup — stop servers, close connections
@Disabled("reason")	Skips this test or class	Always include reason and ticket number
@DisplayName("text")	Human-readable name in reports	Makes CI failures self-documenting
@ParameterizedTest	Run test with multiple inputs	Combine with @ValueSource/@CsvSource/@MethodSource
@ValueSource	Array of single values	strings, ints, longs, doubles, booleans, chars
@CsvSource	Multiple params as CSV strings	Input-output pairs, readable inline data
@MethodSource	Factory method provides arguments	Complex objects, large datasets, dynamic data
@NullSource / @EmptySource	Add null / empty argument	Test null and empty edge cases separately
@Nested	Inner class as a test group	Group related tests, improve readability
@ExtendWith(X.class)	Add JUnit 5 extension	Required for MockitoExtension, SpringExtension

Mockito Cheat Sheet

Operation	Syntax	When to Use
Create mock	@Mock / Mockito.mock(Class.class)	For interfaces and classes with dependencies
Create spy	@Spy / Mockito.spy(realObj)	Wrap real object, override selective methods
Inject mocks	@InjectMocks	Create SUT and inject @Mock/@Spy fields
Stub return	when(mock.method(arg)).thenReturn(val)	Configure what mock returns
Stub throw	when(mock.method()).thenThrow(new Ex())	Simulate failures and error conditions
Dynamic stub	when(mock.m()).thenAnswer(inv -> ...)	Return computed value based on argument
Stub void	doNothing().when(mock).method()	For void methods only
Verify once	verify(mock).method()	Default — called exactly once

Verify n times	<code>verify(mock, times(n)).method()</code>	Specific number of invocations
Verify never	<code>verify(mock, never()).method()</code>	Assert method was NOT called
Capture arg	<code>verify(m).method(captor.capture()); captor.getValue()</code>	Inspect what was passed to mock
Any matcher	<code>any(), anyLong(), anyString()</code>	Match any argument of that type
Exact matcher	<code>eq(value)</code>	Used when mixing with other matchers
In-order verify	<code>InOrder io = inOrder(m1,m2); io.verify(m1).a(); io.verify(m2).b()</code>	Verify execution sequence

TIP

FINAL THOUGHT: Unit testing with JUnit 5 and Mockito is not about achieving 100% code coverage — it is about having confidence that your code does exactly what it is supposed to do, and that you will know immediately when it stops doing so. A well-written test suite is your safety net for refactoring, your documentation of behavior, and your first line of defense against regressions. Write tests as carefully as you write production code. Your future self will thank you.

